Routing, Permissions, and WebSockets

WebSockets, runtime permissions, and routing with go_router

Dinu-Ștefan Rusu ADMD

What you should leave with

Realtime transport

Which of WebSocket, SSE, long polling and MQTT fits a problem, and the handshake that opens a socket.

Reliable connections

Backoff, heartbeats, wss, and what the OS does to a socket when the app leaves the screen.

Runtime permissions

Manifest and plist entries, Android 13+ granular media, and every state permission_handler returns.

Routing with go_router

Routes as data, parameters, nested shells, and a redirect guard wired to auth state.

1 Realtime connections

WebSockets, SSE, long polling, MQTT

Contents Page

1 **Realtime connections** 3
2 Runtime permissions 13
3 Declarative routing 22

HTTP polling versus WebSockets

HTTP is request and response: the server can never speak first. Apps fake push by polling, paying for a request almost every time.

	Polling over HTTP	WebSocket
Connection	a request per poll, or a reused socket	one socket, held open
Who may speak	the client only	either side, any moment
Worst-case latency	one full poll interval	one network hop
Cost while idle	a request every interval, forever	a few bytes of keepalive

The handshake: HTTP 101 Switching Protocols

```
GET /chat HTTP/1.1
Host: realtime.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZQ==
Sec-WebSocket-Version: 13

HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzw==

// same TCP connection, now WebSocket frames
```

What just happened

A WebSocket starts as an ordinary HTTP request, so it passes through the same proxies and firewalls as HTTPS.

The 101 response ends the HTTP part. The same TCP connection now carries binary WebSocket frames (RFC 6455). The opening request can carry cookies and a query string, which is how the socket gets authenticated.

Push transports compared

Transport	Direction	Runs over	Reach for it when
WebSocket	full duplex	TCP, HTTP upgrade	both sides talk: chat, presence, games
SSE	server to client	one long HTTP response	a one-way feed; reconnect comes free
Long polling	server to client	ordinary HTTP requests	a restrictive network blocks everything else
MQTT	pub/sub	TCP or WebSocket	devices: tiny frames, retained messages

In MQTT, a broker fans one reading out to every subscriber. MEC Lecture 4 covers the broker side.

web_socket_channel: connection lifecycle

```dart
// connect() is lazy; wss:// only, always.
final channel = WebSocketChannel.connect(
  Uri.parse('wss://realtime.example.com/chat'),
);
await channel.ready;  // throws on failure
final sub = channel.stream.listen(
  (data) => onMessage(data as String),
  onError: (e, st) => scheduleReconnect(),
  onDone: () => scheduleReconnect(),
);
channel.sink.add(jsonEncode({'t': 'msg'}));
await sub.cancel();
await channel.sink.close(status.goingAway);
```

The two lines everyone forgets

`await channel.ready`: connect() returns instantly and never throws. Without it, a rejection surfaces three screens later as a stream error. `sink.close()` in dispose(): a socket is not garbage collected. Left open, it keeps a radio awake and a server slot held.

Transport encryption with wss://

Every frame crosses the network in the clear.

Messages and ids are readable on shared Wi-Fi, and any middlebox can rewrite them. wss:// is the same protocol carried inside TLS, on port 443.

iOS App Transport Security and Android's cleartext policy both block ws:// by default. Do not add a platform exception to work around it.

A socket is authenticated once, then trusted for hours.

The server checks the token at the upgrade request, and checks it again when it expires.

Short-lived tokens, re-checked. A socket opened at nine with a one-hour token must not still be trusted at noon. The server closes it, and the client reconnects with a fresh one.

WebSocket authentication

Query string

`?token=...` is simplest and works everywhere, but it lands in server access logs.

Subprotocol header

Sec-WebSocket-Protocol carries the credential instead. It is not logged, but some proxies handle it awkwardly.

First message

Connect anonymously, send an auth frame immediately, and have the server close the socket if it does not arrive within a second.

Pick the one your server and proxies support, and keep the token short-lived.

Heartbeats and dead connections

TCP never tells you the peer is gone: only that it sent a FIN, and a phone out of Wi-Fi range sends neither.
Carrier NAT and a Wi-Fi to LTE handover change your address mid-connection. Keepalive defaults to two hours: far too slow.

```
// ping, expect a reply
beat(_) {
  if (_awaitingPong) {
    return reconnect();
  }
  _awaitingPong = true;
  channel.sink.add(ping);
}
```

A half-open connection is worse than closed. A ”sent” message can sit in a kernel buffer while the UI shows it delivered.

Reconnection with backoff and jitter

```
// Never reconnect in a tight loop.
Duration _backoff() {
  const base = Duration(seconds: 1);
  const cap = Duration(minutes: 2);
  final n = base * math.pow(2, _attempt);
  final ms = (n > cap ? cap : n).inMilliseconds;
  return Duration(milliseconds: _r.nextInt(ms));
}
void scheduleReconnect() {
  if (_disposed) return;
  _attempt++;
  _timer = Timer(_backoff(), _connect);
}
```

Why jitter, not just doubling

Ten thousand clients retry at the same instant: a thundering herd. Full jitter spreads them evenly. Reset _attempt on a connection that lasts, not one that merely opens.

Sockets in the background

A WebSocket is a foreground affordance. iOS suspends a backgrounded app within seconds; no close frame is sent, the server just stops hearing from you. Android does the same more gradually, through Doze.

Anything that must reach the user while away is a push notification, FCM or APNs, not a frame on the stream.

Paused
close the socket yourself, cleanly

Backgrounded
FCM or APNs carries what matters

Resumed
reconnect, then resync what was missed

A common realtime bug on mobile. It works foreground and breaks the moment the phone locks. Give every message a server-side id so resync is a gap query.

2

Runtime permissions

Manifest, Info.plist, permission_handler statuses

Contents Page

1 Realtime connections 3
2 Runtime permissions 13
3 Declarative routing 22

Declaring and requesting permissions

Step one is static: you declare, at build time, what the app might ever ask for, in the manifest or Info.plist.
Step two is at runtime: the user is asked once, in a dialog you cannot style or reword, and can say no.

Declare
AndroidManifest.xml and Info.plist

Ask, in context
permission_handler's request()

Handle every answer
including the one you cannot retry

Skip step one and step two does not fail gracefully. Android silently denies an undeclared permission forever. iOS terminates the app on first touch, and review rejects the build.

Android: AndroidManifest.xml

```
<!-- android/app/src/main/AndroidManifest.xml -->
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
  <uses-permission android:name="android.permission.CAMERA"/>
  <uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>
  <uses-permission android:name="android.permission.READ_MEDIA_IMAGES"/>
  <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"
                   android:maxSdkVersion="32"/>
  <uses-feature android:name="android.hardware.camera"
                android:required="false"/>
</manifest>
```

Declaring is not requesting: these entries only make the runtime request legal. `maxSdkVersion` keeps the old storage permission for old phones, off on Android 13+. Flutter merges every plugin's manifest into yours at build time.

iOS: Info.plist

```
<!-- ios/Runner/Info.plist -->
<key>NSCameraUsageDescription</key>
<string>MapChat uses the camera to send photos in a chat.</string>
<key>NSMicrophoneUsageDescription</key>
<string>MapChat records a voice note while you hold the button.</string>
<key>NSLocationWhenInUseUsageDescription</key>
<string>MapChat shows friends near you while the app is open.</string>
<key>NSPhotoLibraryUsageDescription</key>
<string>Pick a photo you already have and send it.</string>
```

That sentence is printed inside the system dialog: the only explanation the user gets, once. Say what the user gains, not what the app needs. A missing key is a hard crash, not a warning: the app is terminated on first touch.

Android 13+ granular media permissions

One coarse ”read everything” permission became several narrow ones, and notifications stopped being granted at install.

Capability	Before: API 32	Android 13+ (API 33+)
Photos	READ_EXTERNAL_STORAGE	READ_MEDIA_IMAGES
Video	READ_EXTERNAL_STORAGE	READ_MEDIA_VIDEO
Audio and music	READ_EXTERNAL_STORAGE	READ_MEDIA_AUDIO
Notifications	granted at install	POST_NOTIFICATIONS prompt
Part of the library	not possible	user-selected, API 34+

POST_NOTIFICATIONS is the one that matters most here. Your FCM push silently delivers nothing on an Android 13+ phone until the user grants it at runtime.

Permission statuses

State	What it means	What you do
isGranted	you may call the API	proceed
isDenied	refused once; Android allows asking again	rationale, request() again
isPermanentlyDenied	Android ”don’t ask again”; iOS any refusal	request() is a no-op, use openAppSettings()
isRestricted	policy blocked: parental controls, MDM	hide the feature
isLimited	a picked subset: iOS Photos, Android 14+	treat as granted
isProvisional	iOS notifications delivered quietly	ask to upgrade later

Limited and provisional access

isLimited: the user did not say no.

On iOS, a Photos request can return a user-picked subset of the library instead of everything. Android 14+ has the same idea for one-time media selection. The picker still works; the app only sees what was chosen.

isProvisional: already delivering, quietly.

iOS can grant notifications with no system prompt at all. They arrive in the notification center without a sound or a banner, until the user upgrades them from inside the app.

A plain isGranted check throws both away. Treat isLimited as granted and offer ”select more”. Treat isProvisional as granted and ask for the upgrade later, not on launch.

permission_handler and permanent denial

```dart
Future<bool> ensureCamera(c) async {
  var s = await Permission.camera.status;
  if (s.isGranted || s.isLimited) {
    return true;
  }
  if (s.isDenied) {
    if (!await rationale(c)) return false;
    s = await Permission.camera.request();
  }
  if (s.isPermanentlyDenied ||
      s.isRestricted) return false;
  return s.isGranted || s.isLimited;
}
```

Three traps

permanentlyDenied only means something after asking once; a launch check reports denied instead. On permanentlyDenied, confirm with the user, then call openAppSettings() and re-read status on resume.

Permission request guidelines

Ask at the moment of need

Never on launch. Ask when the user taps the camera button.

Show your own screen first

A dismissible sheet before the system dialog. Nothing is spent if they say no.

Degrade, don’t dead-end

Camera denied? Offer the gallery. The feature still has to work.

Give them a way back

When permanently denied, offer a button that opens Settings.

Ask for the narrower one

Coarse location and read-media-images are granted more often.

Never ask what you do not use

Reviewers check declared permissions against behavior.

3

Declarative routing

go_router, route parameters, ShellRoute, redirect guards

Contents Page

1 Realtime connections 3
2 Runtime permissions 13
3 Declarative routing 22

Navigator 1.0 and its limits

```dart
// Fine for a local dialog or sheet.
Navigator.of(context).push(
  MaterialPageRoute(
      builder: (_) => ChatScreen(id: id)),
);
// Named routes: no longer recommended.
MaterialApp(routes: {
  '/chat': (_) => const ChatScreen(),
});
Navigator.pushNamed(context, '/chat');
```

What it cannot do

Deep links: a notification that opens /chat/42 becomes a sequence of pushes, by hand, in order.

Whole-stack changes: signing out and clearing every screen behind the user is awkward. The web: named routes do not become URLs, so the address bar and back button are wrong.

go_router is the Flutter team's answer. Navigator 2.0 fixed all three, but its raw API was hard to use directly. go_router implements it for you with a URL-shaped API on top.

go_router route configuration

```
final router = GoRouter(
  initialLocation: '/',
  routes: [
    GoRoute(path: '/',
        builder: (c, s) => Home()),
    GoRoute(
      path: '/chat/:id',
      builder: (c, s) => ChatScreen(
        roomId: s.pathParameters['id']!,
      )),
  ],
  errorBuilder: (c, s) => NotFound(),
);
```

Stack equals a function of state

You declare what exists, instead of saying ”go here”. One `MaterialApp.router` line wires it in, then it works from a URL bar. `context.go()` replaces the stack. `context.push()` adds to it.

Passing data to a route

Kind	Navigate with	Read it back as	Survives?
Path param	`go('/chat/42')`	pathParameters['id']	yes
Query param	`go('/search?q=x')`	uri.queryParameters['q']	yes
Extra	`go(path, extra: room)`	extra as Room	no

All three read back through `GoRouterState`, the second argument every builder and redirect callback receives.

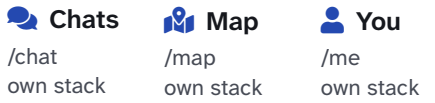
Pass ids, not objects. extra is an in-memory pointer: it does not survive a cold start from a notification or a browser refresh. Put the id in the URL and look the object up on the other side.

ShellRoute and persistent navigation

A bottom navigation bar is a frame around screens. ShellRoute gives a nested Navigator: the Scaffold and its bar are built once, and only the body swaps.

StatefulShellRoute.indexedStack goes further. Each branch keeps its own navigation stack and its own scroll position.

Go three screens deep in Chats, switch to Map and come back, and Chats is still three screens deep, at the same scroll position.



Never nest a Navigator by hand. The shell already is one, and reimplementing this by hand is a common source of navigation bugs.

redirect: an app-wide auth guard

```dart
final router = GoRouter(
  refreshListenable: authRefresh,
  redirect: (context, state) {
    final user =
        FirebaseAuth.instance.currentUser;
    final atLogin =
        state.matchedLocation == '/login';
    if (user == null && !atLogin) {
      return '/login'; }
    if (user != null && atLogin) return '/';
    return null;
  },
);
```

The bridge back to Lecture 10

redirect runs before every navigation, first one included. One place decides who sees what.

authRefresh wraps authStateChanges() in a ChangeNotifier, so sign-out re-runs the guard. Return null to allow, a location to send them elsewhere.

Worked example: nearby chat

1 · Permission

The user taps ”Nearby”. Show your own rationale, request location, and handle limited, denied and permanently denied before drawing a map.

2 · Route

They tap a friend. context.go(’/chat/alice’) builds the screen. redirect already checked they are signed in.

3 · Socket

ChatScreen opens wss://…/chat with a token, listens, sends, heartbeats, reconnects with jitter, and closes cleanly in dispose().

Summary

1. **A WebSocket is one held-open socket.** Keep it alive with wss, heartbeats and jittered backoff, and close it in dispose().
2. **Declaring a permission is not requesting it.** Both platforms punish the gap: silent denial on Android, termination on iOS.
3. **A route is a location, not a sequence of pushes.** go_router turns state into a stack, and one redirect guards all of it.

Further reading

web_socket_channel · permission_handler · go_router · RFC 6455

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel